

MediConnect

Healthcare Appointment & Management Platform

Software Requirements Specification (SRS)

User Story Format | Agile Sprint Documentation

Version 1.0 | December 2025 – May 2026

Author: **Harshvardhan Mundhra** | Systems Engineer Intern, Infosys Limited

Tech Stack: Spring Boot · Angular 17 · MySQL · Redis · JWT/RBAC · WebRTC · Razorpay
· NgRx · JUnit 5

Table of Contents

1. Project Overview	3
2. System Architecture & Tech Stack	4
3. Roles & Permissions	4
4. Epic 1: Authentication & Authorization	5
5. Epic 2: Patient Management	6
6. Epic 3: Doctor Management	7
7. Epic 4: Appointment Booking	8
8. Epic 5: Video Consultation (WebRTC)	9
9. Epic 6: Digital Health Vault	10
10. Epic 7: Prescription & Pharmacy	11
11. Epic 8: Payment Integration (Razorpay)	12
12. Epic 9: Admin Analytics Dashboard	13
13. Epic 10: Notifications	14
14. Non-Functional Requirements	15
15. Testing Strategy	16

1. Project Overview

MediConnect is a production-grade **multi-role healthcare appointment and management platform** developed as part of a 6-month Infosys internship. The system supports four primary roles — **Patient, Doctor, Pharmacy, and Admin** — enabling end-to-end healthcare workflows including in-person and video consultations, digital health vault management, pharmacy inventory tracking, prescription allergy checks, and an admin analytics dashboard.

1.1 Project Goals

Goal	Description
Multi-role Access	Distinct dashboards and workflows per role with RBAC enforcement
Scalable Microservices	Modular Spring Boot services with Redis caching, circuit breakers, paginated APIs (p95 < 300 ms)
Accessible UI	WCAG 2.1 AA compliant Angular 17 PWA with Angular CDK and Material
Secure Transactions	Razorpay sandbox payments, JWT stateless auth, OAuth 2.0, Spring Security
Observability	SonarQube CI gates, 80%+ JUnit 5/Mockito coverage, Swagger/OpenAPI docs

2. System Architecture & Tech Stack

2.1 Architecture Overview

- **Angular 17 Monorepo (Frontend):** Standalone components, lazy-loaded feature modules, route-level guards, NgRx Store + Effects, RxJS Observables, SSE for real-time updates, Angular Material with custom design tokens.
- **Spring Boot Microservices (Backend):** RESTful APIs, JWT/RBAC stateless authentication, Spring Security, Redis caching, circuit-breaker patterns (Resilience4j), paginated responses.
- **Database Layer:** MySQL (primary persistence), Redis (caching / session), ABAP Data Dictionary patterns for schema design.
- **External Integrations:** Razorpay (payments), Jitsi/WebRTC (video), SonarQube (CI quality gates).
- **API Documentation:** Swagger / OpenAPI 3.0 for all service endpoints.

2.2 Technology Matrix

Layer	Technologies
Frontend	Angular 17, TypeScript, Angular Material, Angular CDK, NgRx, RxJS, Chart.js, PWA
Backend	Spring Boot, Spring Security, Spring REST, JWT, OAuth 2.0, Resilience4j, SSE
Database	MySQL, Redis (caching), Hibernate JPA
Testing	JUnit 5, Mockito, SonarQube, Swagger/OpenAPI
Payments	Razorpay Sandbox (order API, webhook verification)
Video	WebRTC via Jitsi Meet SDK
DevOps	GitHub, VS Code, Postman, SonarQube CI pipeline

3. Roles & Permissions

Role	Key Capabilities	Guard
PATIENT	Book appointments, video consultations, manage health vault, view prescriptions, make payments	PatientGuard
DOCTOR	Manage schedule, view patient history, write prescriptions, allergy-check logic, join video calls	DoctorGuard
PHARMACY	Manage inventory, process prescriptions, receive low-stock alerts	PharmacyGuard
ADMIN	User management, revenue analytics, compliance dashboard, system configuration	AdminGuard

4. Epic 1: Authentication & Authorization

All authentication is stateless using **JWT tokens** with role-based claims. Spring Security enforces method-level access. Angular route guards redirect unauthorized users. Refresh tokens are stored securely; access tokens expire in 15 minutes.

US-001

Register an account with email and password

Priority: **Must-Have**

SP: **3**

As a [New User](#), I want to register an account with email and password, **so that** I can access the platform with my assigned role.

Acceptance Criteria:

- Registration form validates email uniqueness in real time (debounce 400 ms)
- Password must meet strength policy (8+ chars, uppercase, number, special char)
- Role is selected during registration (Patient / Doctor / Pharmacy)
- On success, system sends verification email; unverified users cannot log in
- Duplicate email returns HTTP 409 with descriptive error message

Technical Notes:

- BCrypt password hashing (strength=12) via Spring Security PasswordEncoder
- Angular reactive form with custom PasswordStrengthValidator
- Email uniqueness checked via /api/auth/check-email before submission

API Endpoints:

```
POST /api/auth/register
GET /api/auth/check-email?email={email}
```

US-002

Log in with email and password

Priority: **Must-Have**

SP: **2**

As a [Registered User](#), I want to log in with email and password, **so that** I receive a JWT token and am redirected to my role-specific dashboard.

Acceptance Criteria:

- Successful login returns access token (15 min) + refresh token (7 days)
- Failed login shows generic error (no user enumeration)
- After 5 failed attempts, account is locked for 15 minutes
- JWT payload contains userId, role, email, and iat/exp claims
- Angular stores tokens in memory (access) and HttpOnly cookie (refresh)

API Endpoints:

```
POST /api/auth/login
POST /api/auth/refresh
POST /api/auth/logout
```

US-003

Have my session automatically refreshed

Priority: **Must-Have**

SP: **2**

As a [Authenticated User](#), I want to have my session automatically refreshed, **so that** I don't get logged out while actively using the platform.

Acceptance Criteria:

- HTTP interceptor silently refreshes access token when expiry < 2 minutes
- If refresh token is expired/invalid, user is redirected to login
- Concurrent refresh requests are queued; only one refresh call is made

Technical Notes:

- Angular HTTP interceptor with BehaviorSubject token queue

US-004

Manage user accounts (activate/deactivate/reset password)

Priority: **Must-Have**
SP: **3**

As a **Admin**, I want to manage user accounts (activate/deactivate/reset password), so that I can maintain platform security and user access.

Acceptance Criteria:

- Admin can search users by name, email, or role
- Admin can deactivate/reactivate accounts with reason
- Admin can trigger password reset email
- All admin actions are audit-logged with timestamp and admin ID

API Endpoints:

```
GET /api/admin/users
PATCH /api/admin/users/{id}/status
POST /api/admin/users/{id}/reset-password
```

5. Epic 2: Patient Management

US-005

Complete my health profile with medical history

Priority: **Must-Have**
SP: 3

As a **Patient**, I want to complete my health profile with medical history, so that doctors can review my background before consultations.

Acceptance Criteria:

- Profile includes: blood type, allergies, chronic conditions, current medications, emergency contact
- Allergies list is used for prescription allergy-check logic by doctors
- Profile completion percentage displayed; reminder shown below 60%
- All data encrypted at rest; only treating doctors can view full profile

Technical Notes:

- Allergy list stored as normalised JSON array in MySQL with indexed lookup
- Angular CDK FocusTrap ensures modal dialogs are keyboard accessible

API Endpoints:

```
GET /api/patients/{id}/profile
PUT /api/patients/{id}/profile
```

US-006

Search for doctors by specialty, location, and availability

Priority: **Must-Have**
SP: 3

As a **Patient**, I want to search for doctors by specialty, location, and availability, so that I can find the most suitable doctor for my needs.

Acceptance Criteria:

- Search supports specialty filter, city/area filter, and available date filter
- Results are paginated (10/page) with infinite scroll
- Doctor cards show name, photo, specialty, rating, next available slot
- Search results cached in Redis (TTL 5 min); invalidated on schedule change
- p95 response time < 300 ms for search queries

API Endpoints:

```
GET /api/doctors/search?specialty=&location;=&date;=&page;=&size;=
```

US-007

View my appointment history and upcoming appointments

Priority: **Must-Have**
SP: 2

As a **Patient**, I want to view my appointment history and upcoming appointments, so that I can track my healthcare journey.

Acceptance Criteria:

- Upcoming appointments sorted by date/time ascending
- Past appointments show status: Completed, Cancelled, No-Show
- Patient can cancel upcoming appointment (up to 2 hours before slot)
- Cancellation reason is required; cancellation policy shown
- Real-time status updates pushed via SSE stream

API Endpoints:

```
GET /api/patients/{id}/appointments?status=upcoming|past&page;=&size;=
PATCH /api/appointments/{id}/cancel
```

US-008

Rate and review a completed consultation

Priority: **Should-Have**
SP: 2

As a **Patient**, I want to rate and review a completed consultation, so that I can help other patients choose doctors.

Acceptance Criteria:

- Rating prompt appears 1 hour after appointment end
- Rating: 1-5 stars + optional text review (max 500 chars)
- Each appointment can be rated only once
- Reviews moderated by admin before going public
- Aggregate rating recalculated after each review

API Endpoints:

POST /api/appointments/{id}/review

GET /api/doctors/{id}/reviews

6. Epic 3: Doctor Management

US-009

Set up my availability schedule

Priority: **Must-Have**
SP: 3

As a **Doctor**, I want to set up my availability schedule, so that patients can book appointments during my working hours.

Acceptance Criteria:

- Doctor sets recurring weekly schedule (days + time slots, 30/60 min increments)
- Doctor can block individual dates (vacation, public holiday)
- Schedule changes propagate immediately; existing bookings are preserved
- Frontend uses Angular CDK drag-and-drop for visual schedule builder
- Conflicts (overlapping slots) are validated server-side and rejected with HTTP 409

API Endpoints:

```
GET /api/doctors/{id}/schedule
PUT /api/doctors/{id}/schedule
POST /api/doctors/{id}/schedule/block
```

US-010

View patient health profile before consultation

Priority: **Must-Have**
SP: 2

As a **Doctor**, I want to view patient health profile before consultation, so that I can prepare an informed treatment plan.

Acceptance Criteria:

- Doctor can view patient profile only for confirmed/upcoming appointments
- Profile access logged for audit trail
- Allergy list highlighted prominently; conflict alerts shown when prescribing
- Previous prescriptions from MediConnect visible with date and doctor name

API Endpoints:

```
GET /api/doctors/{doctorId}/patients/{patientId}/profile
```

US-011

Write and send prescriptions to pharmacy

Priority: **Must-Have**
SP: 5

As a **Doctor**, I want to write and send prescriptions to pharmacy, so that patients can receive medication without visiting the clinic.

Acceptance Criteria:

- Prescription form includes: diagnosis, medications (name/dose/frequency/duration), notes
- System performs allergy check against patient's allergy list before submission
- Allergy conflict shows modal with specific drug-allergy pair; doctor must acknowledge
- Prescription sent to patient's preferred pharmacy automatically
- Prescription PDF generated and added to patient's health vault
- Doctor can mark prescription as shared (visible to other treating doctors)

Technical Notes:

- Allergy check via POST /api/prescriptions/allergy-check with drug-allergy matrix
- PDF generated server-side using Apache PDFBox; stored in health vault
- Prescription state machine: DRAFT → SUBMITTED → DISPENSED → CANCELLED

API Endpoints:

```
POST /api/prescriptions
POST /api/prescriptions/allergy-check
GET /api/prescriptions/{id}/pdf
```

US-012

View earnings and consultation analytics

Priority: **Should-Have**
SP: **3**

As a Doctor, I want to view earnings and consultation analytics, **so that** I can track my performance and income.

Acceptance Criteria:

- Dashboard shows total consultations, revenue, avg rating, and cancellation rate
- Charts filter by: last 7 days, last 30 days, custom date range
- Revenue breakdown by consultation type (in-person vs video)
- Charts built with Chart.js rendered in Angular component

API Endpoints:

```
GET /api/doctors/{id}/analytics?from=&to;=
```

7. Epic 4: Appointment Booking

US-013

Book an in-person or video appointment

Priority: **Must-Have**
SP: 5

As a **Patient**, I want to book an in-person or video appointment, so that I can get the consultation I need at a convenient time.

Acceptance Criteria:

- Booking flow: Select Doctor → Choose Type → Pick Slot → Confirm → Pay
- Available slots fetched in real time; booked slots hidden immediately (optimistic locking)
- Patient selects consultation type: In-Person or Video Call
- Payment required to confirm booking (Razorpay integration — see Epic 8)
- Confirmation email sent with appointment summary and join link (for video)
- Concurrent booking conflict resolved via database-level row locking

Technical Notes:

- Slot reservation: 10-minute hold during payment; released if payment times out
- Pessimistic lock: SELECT ... FOR UPDATE on slot record during booking
- Angular stepper component for multi-step booking flow

API Endpoints:

```
GET /api/appointments/slots?doctorId=&date;=  
POST /api/appointments/reserve  
POST /api/appointments/confirm
```

US-014

Receive appointment reminders

Priority: **Must-Have**
SP: 2

As a **Patient / Doctor**, I want to receive appointment reminders, so that I don't miss upcoming consultations.

Acceptance Criteria:

- Email reminder sent 24 hours before appointment
- In-app notification sent 30 minutes before appointment via SSE
- Reminders include: doctor name, date/time, type, and video join link
- Patient can reschedule from reminder email (up to 2 hours before)

API Endpoints:

```
POST /api/notifications/appointment-reminder (internal scheduler)
```

US-015

View and manage appointment queue for today

Priority: **Must-Have**
SP: 3

As a **Doctor / Admin**, I want to view and manage appointment queue for today, so that consultations run smoothly and on time.

Acceptance Criteria:

- Today's appointments listed in chronological order with status badge
- Doctor can mark appointment as: In Progress, Completed, No-Show
- Admin can see all appointment statuses across all doctors
- Status changes broadcast via SSE to all relevant parties
- Next patient automatically highlighted when current is marked complete

API Endpoints:

```
GET /api/appointments/today?doctorId=  
PATCH /api/appointments/{id}/status
```

8. Epic 5: Video Consultation (WebRTC / Jitsi)

US-016

Join a video consultation from my appointment

Priority: **Must-Have**
SP: 5

As a **Patient**, I want to join a video consultation from my appointment, **so that** I can have a remote consultation without visiting the clinic.

Acceptance Criteria:

- Video call uses Jitsi Meet SDK embedded in Angular component
- Join button activates 5 minutes before scheduled appointment time
- Unique room ID generated per appointment; only booked patient and doctor can join
- In-call features: mute/unmute, camera on/off, screen share, chat, end call
- Call duration tracked; doctor cannot write prescription until call is completed
- Waiting room shown if doctor has not joined yet

Technical Notes:

- Jitsi room token JWT signed by backend with appointmentId as room name
- Angular component wraps Jitsi IFrame API with event listeners
- Call status persisted: SCHEDULED → WAITING → IN_PROGRESS → ENDED

API Endpoints:

```
GET /api/appointments/{id}/video-token
POST /api/appointments/{id}/video-status
```

US-017

Access patient medical summary during video call

Priority: **Must-Have**
SP: 3

As a **Doctor**, I want to access patient medical summary during video call, **so that** I have all necessary context during remote consultation.

Acceptance Criteria:

- Side panel shows patient profile, allergies, current medications, past visits
- Doctor can take consultation notes in the side panel (auto-saved every 30 s)
- Prescription form accessible from within video call interface
- Notes saved as part of consultation record after call ends

Technical Notes:

- Auto-save via Angular FormControl valueChanges + debounceTime(30000)
- Notes stored in consultation_notes table linked to appointment_id

9. Epic 6: Digital Health Vault

US-018

Upload and manage my medical documents

Priority: **Must-Have**
SP: 5

As a **Patient**, I want to upload and manage my medical documents, so that I have all my health records in one secure place.

Acceptance Criteria:

- Supported file types: PDF, JPG, PNG, DICOM (max 20 MB per file)
- Documents categorised: Lab Reports, Prescriptions, Imaging, Insurance, Other
- Patient can add tags and description to each document
- Documents displayed in a paginated grid with search and filter by date/category
- Document is soft-deleted; admin can restore within 30 days
- Total storage quota per patient: 500 MB (configurable by admin)

Technical Notes:

- Files stored in server filesystem (production: S3-compatible bucket)
- Virus scan on upload using ClamAV (configurable)
- File metadata in MySQL; blob in filesystem/object-store

API Endpoints:

```
POST /api/vault/upload
GET  /api/vault?page=&size=&category=&search;=
DELETE /api/vault/{id}
```

US-019

Share specific documents with a doctor

Priority: **Must-Have**
SP: 3

As a **Patient**, I want to share specific documents with a doctor, so that treating doctors can access my records securely.

Acceptance Criteria:

- Patient selects document(s) and chooses a doctor to share with
- Share creates a time-limited access grant (default: 30 days, configurable)
- Shared documents appear in doctor's patient profile view
- Patient can revoke access at any time
- Sharing activity visible in audit log

API Endpoints:

```
POST /api/vault/{id}/share
GET  /api/vault/shared-with-me
DELETE /api/vault/share/{shareId}
```

US-020

Automatically add prescription PDFs to patient's health vault

Priority: **Must-Have**
SP: 2

As a **System**, I want to automatically add prescription pdfs to patient's health vault, so that patients always have their prescriptions accessible.

Acceptance Criteria:

- On prescription submission, PDF is auto-generated and saved to vault
- Document tagged as 'Prescription' with doctor name and date
- Patient receives in-app notification when new prescription is added
- PDF includes: patient name, doctor name, date, diagnosis, medications, doctor signature

10. Epic 7: Prescription & Pharmacy Management

US-021

View incoming prescriptions for my pharmacy

Priority: **Must-Have**
SP: 3

As a **Pharmacy Staff**, I want to view incoming prescriptions for my pharmacy, so that I can prepare medication before the patient arrives.

Acceptance Criteria:

- Pharmacy dashboard shows queue of pending prescriptions with patient name and time
- Prescriptions sorted by urgency (marked by doctor) and submission time
- Pharmacy can view full prescription details including allergy notes
- Status tracking: PENDING → PREPARING → READY FOR PICKUP → DISPENSED
- Patient receives SMS notification when prescription is READY FOR PICKUP

API Endpoints:

```
GET /api/pharmacy/prescriptions?status=&page;=  
PATCH /api/pharmacy/prescriptions/{id}/status
```

US-022

Manage medication inventory

Priority: **Must-Have**
SP: 5

As a **Pharmacy Staff**, I want to manage medication inventory, so that I can maintain adequate stock and prevent medication shortages.

Acceptance Criteria:

- Inventory lists all medications with: name, generic name, stock quantity, unit, expiry date
- Low-stock alerts triggered when quantity falls below configurable threshold
- Alert sent to pharmacy staff via in-app notification and email
- Pharmacy can update stock quantities (add/remove with reason)
- Expiry alerts generated 30 days before medication expires
- Inventory report exportable to CSV

Technical Notes:

- Low-stock check triggered on every stock update via @EventListener
- Threshold configured per-medication or globally in admin settings

API Endpoints:

```
GET /api/pharmacy/inventory?search=&lowStock;=  
PATCH /api/pharmacy/inventory/{id}  
GET /api/pharmacy/inventory/export/csv
```

US-023

Check drug-allergy interactions before prescribing

Priority: **Must-Have**
SP: 5

As a **Doctor**, I want to check drug-allergy interactions before prescribing, so that I avoid prescribing medications that could harm the patient.

Acceptance Criteria:

- Allergy check performed automatically when drug is added to prescription
- Check compares drug (and its class) against patient's registered allergy list
- Red warning modal shown with: drug name, allergy name, severity level
- Doctor must explicitly acknowledge the warning before proceeding
- Warning acknowledgement logged with doctor ID and timestamp
- System does not block submission — final decision rests with doctor

Technical Notes:

- Drug-allergy matrix maintained in drug_interactions table
- Angular AllergySafetyModalComponent uses CDK Dialog with FocusTrap

11. Epic 8: Payment Integration (Razorpay)

US-024

Pay for an appointment using Razorpay

Priority: **Must-Have**
SP: 5

As a [Patient](#), I want to pay for an appointment using razorpay, **so that** my booking is confirmed after successful payment.

Acceptance Criteria:

- Razorpay order created server-side before checkout modal opens
- Razorpay checkout modal embedded in Angular using Razorpay JS SDK
- On payment success: Razorpay sends webhook → backend verifies signature → appointment confirmed
- On payment failure: slot reservation released; patient prompted to retry
- Payment receipt emailed to patient; PDF saved to health vault
- All transactions stored in payments table with Razorpay order_id and payment_id
- Refund processing for cancellations initiated via Razorpay Refund API

Technical Notes:

- HMAC-SHA256 signature verification on webhook using Razorpay secret
- Idempotent payment confirmation: duplicate webhook events ignored
- Payment amount calculated server-side; never trusted from frontend

API Endpoints:

```
POST /api/payments/create-order
POST /api/payments/webhook (Razorpay webhook endpoint)
POST /api/payments/{id}/refund
GET /api/payments/{id}/receipt
```

US-025

View payment and revenue reports

Priority: **Must-Have**
SP: 3

As a [Admin](#), I want to view payment and revenue reports, **so that** I can track platform financial health.

Acceptance Criteria:

- Revenue dashboard shows total, daily, weekly, monthly revenue
- Breakdown by consultation type, doctor specialty, and payment status
- Failed payment rate tracked and displayed
- Revenue data exportable to CSV/Excel
- Charts powered by Chart.js with Angular reactive data streams

API Endpoints:

```
GET /api/admin/payments/report?from=&to;=&groupBy;=
```

12. Epic 9: Admin Analytics Dashboard

US-026

View platform-wide analytics on a real-time dashboard

Priority: **Must-Have**
SP: 5

As a [Admin](#), I want to view platform-wide analytics on a real-time dashboard, so that I can monitor platform health and make data-driven decisions.

Acceptance Criteria:

- KPI tiles: Total Users, Active Doctors, Appointments Today, Revenue Today
- Charts: Daily Active Users (line), Appointments by Type (donut), Revenue Trend (bar), Specialty Distribution (pie)
- All charts filterable by date range (last 7/30/90 days, custom range)
- Real-time data pushed via SSE; auto-refreshes every 60 seconds
- WCAG 2.1 AA: all charts have aria-labels and data tables as alternatives

Technical Notes:

- Angular CDK LiveAnnouncer announces chart updates for screen readers
- Chart.js datasets updated via RxJS BehaviorSubject subscriptions
- Admin dashboard lazy-loaded in separate feature module

API Endpoints:

```
GET /api/admin/analytics/overview?from=&to;=  
GET /api/admin/analytics/stream (SSE)
```

US-027

Manage doctor onboarding and verification

Priority: **Must-Have**
SP: 3

As a [Admin](#), I want to manage doctor onboarding and verification, so that only verified medical professionals can consult on the platform.

Acceptance Criteria:

- Doctors submit: medical registration number, specialization, certificates
- Admin reviews submissions in a verification queue
- Admin can approve, reject (with reason), or request more documents
- Approved doctors receive email confirmation and platform access
- Rejected doctors can resubmit after addressing feedback

API Endpoints:

```
GET /api/admin/doctors/pending-verification  
PATCH /api/admin/doctors/{id}/verification
```


13. Epic 10: Notifications

US-028

Receive real-time in-app notifications

Priority: **Must-Have**
SP: 3

As a **All Users**, I want to receive real-time in-app notifications, **so that** I am immediately informed of important events.

Acceptance Criteria:

- Notifications delivered via Server-Sent Events (SSE) to connected Angular clients
- Notification bell shows unread count badge in navbar
- Notification types: appointment_reminder, prescription_ready, payment_confirmed, new_message, admin_alert
- Clicking notification navigates to relevant section
- Notifications persist in DB; last 50 shown in notification panel
- All notifications marked as read after opening panel

Technical Notes:

- SSE endpoint: GET /api/notifications/stream (user-scoped)
- Angular NotificationService uses EventSource API with reconnect logic
- Spring SseEmitter per connected user, stored in ConcurrentHashMap

API Endpoints:

```
GET /api/notifications/stream
GET /api/notifications?page=&size;=
PATCH /api/notifications/mark-all-read
```

US-029

Configure my notification preferences

Priority: **Should-Have**
SP: 2

As a **All Users**, I want to configure my notification preferences, **so that** I only receive notifications that are relevant to me.

Acceptance Criteria:

- User can toggle: Email notifications, In-app notifications, SMS (if enabled)
- Per-event type toggles (e.g., disable appointment reminders but keep payment alerts)
- Preferences saved in user_preferences table; respected by notification service
- Admin cannot disable critical security notifications (password reset, account lockout)

API Endpoints:

```
GET /api/users/{id}/notification-preferences
PUT /api/users/{id}/notification-preferences
```

14. Non-Functional Requirements

ID	Category	Requirement	Implementation
NFR-001	Performance	API p95 response time < 300 ms under normal load	Redis caching, paginated queries, indexed DB columns
NFR-002	Scalability	Stateless microservices; horizontal scaling via container replicas	JWT stateless auth; no server-side session
NFR-003	Security	OWASP Top 10 mitigated; HTTPS enforced; SQL injection prevention	Spring Security, Hibernate parameterised queries, CORS config
NFR-004	Reliability	Circuit breaker prevents cascade failure	Resilience4j circuit breaker + fallback on all external calls
NFR-005	Accessibility	WCAG 2.1 AA for all Angular UI components	Angular CDK FocusTrap, LiveAnnouncer, Angular Material a11y
NFR-006	Test Coverage	≥ 80% unit test coverage across all Spring Boot services	JUnit 5 + Mockito; SonarQube CI gate blocks merge below threshold
NFR-007	Code Quality	SonarQube quality gate: 0 blocker/critical issues on merge	SonarQube integrated in CI pipeline
NFR-008	API Docs	All endpoints documented with request/response schemas	Swagger / OpenAPI 3.0 via springdoc-openapi
NFR-009	Data Integrity	No orphaned records; cascading deletes enforced	JPA CascadeType.ALL + database FK constraints
NFR-010	Observability	Structured logs for all exceptions and security events	SLF4J + Logback with JSON log format

15. Testing Strategy

15.1 Unit Testing (JUnit 5 + Mockito)

- All Spring Boot service layer methods tested with $\geq 80\%$ line coverage
- Mockito used to mock repository and external service dependencies
- Test naming convention: *methodName_stateUnderTest_expectedBehaviour*
- Parameterised tests for validation rules (e.g., password strength, allergy check matrix)
- Edge cases covered: null inputs, duplicate entries, expired tokens, concurrent requests

15.2 Integration Testing

- Spring Boot `@SpringBootTest` with H2 in-memory DB for repository layer tests
- `MockMvc` used for controller layer testing with JWT mock tokens
- Razorpay webhook signature verification tested with real HMAC signatures
- SSE endpoint tested for correct event delivery using `SseEmitter` mocks

15.3 CI Quality Gates (SonarQube)

- SonarQube connected to GitHub repository; analysis runs on every pull request
- Quality gate blocks merge if: coverage $< 80\%$, blocker/critical issues > 0 , code duplication $> 3\%$
- Security hotspots reviewed and resolved before release
- Technical debt target: < 1 day per sprint

15.4 Test Coverage Summary

Module	Coverage Target	Test Types
Auth Service	$\geq 85\%$	Unit, Integration
Appointment Service	$\geq 85\%$	Unit, Integration, Concurrency
Prescription / Allergy Service	$\geq 90\%$	Unit (parameterised), Integration
Payment Service	$\geq 85\%$	Unit, Webhook Integration
Pharmacy Service	$\geq 80\%$	Unit, Integration
Notification Service	$\geq 80\%$	Unit, SSE Integration
Admin Analytics	$\geq 80\%$	Unit, Integration
Overall Platform	$\geq 80\%$	All above

*This SRS document was authored by **Harshvardhan Mundhra** during the Infosys Systems Engineer Internship (Dec 2025 – May 2026). All user stories follow the standard Agile format and were tracked in sprint backlog. API endpoints follow RESTful conventions; all responses include standard error envelopes.*